



2,000 Step LLM Speedrun (120 minutes)

The problem

You have a training script that trains GPT like LLM from scratch but is deliberately mediocre. This problem will have the same data, same step budget, same parameter budget for everyone attempting.

Make the LLM model as good as you can under those constraints.

Your files (llm_handout.zip)

None

data/

train_corpus.txt mixed English + Hindi text (~7 MB). The ONLY training data.

dev_eval.txt held-out text for YOUR scoring loop

starter/

model.py small GPT, plain PyTorch (~1.3M params)

tokenizer.py byte-level tokenizer (vocab 256)

train.py the mediocre baseline trainer

evaluate.py the official scorer – do not break its

interface

Hard caps (violations disqualify the run)

- **Max 2,000 optimizer steps** for the run that produces your final checkpoint.
- **Max 2,000,000 total parameters** (we count them from your checkpoint).
- **Training data: the provided `train_corpus.txt` only.** Your tokenizer too may be trained only on it.
- **Pure PyTorch + numpy + stdlib.** No pretrained weights of any kind, no `transformers`, no compiled/custom kernels, no mamba-ssm/flash-attn packages. Everything must run on the CPU on our grading machine.
- CPU only. No GPU, no cloud.

Because steps and parameters are capped, a faster laptop doesn't directly help with a better final model. "Train longer" and "make it bigger" are not allowed.



What you MAY change - almost everything

Learning rate and schedule, warmup, optimizers, initialization, weight tying, gradient clipping, batch size / gradient accumulation, sequence length, dropout, normalization, positional encoding, the entire architecture (attention, SSM, hybrid - your call), and the tokenizer (char / byte / BPE - think about what Devanagari does to a byte-level tokenizer's sequence lengths).

Two interface rules: your `tokenizer.load()` must encode arbitrary UTF-8 text (keep a byte fallback), and this command must work, unmodified, inside your submission folder:

None

```
python evaluate.py --checkpoint ckpt.pt --text_file <any_text_file>
```

The score

Bits per byte (bpb) on a hidden text file - same style as your data, never seen by you. Lower is better. Per *byte*, not per token, so tokenizer choices stay comparable across candidates. Run the same scorer yourself on `dev_eval.txt` during the hour.

Deliverables (submission folder)

1. `SUMMARY.html` - a coding agent created html file that summarizes `runlog.md`, `nodes.md`, and your architecture you have created, with all the parameters and why you think yours is the best metric in terms of what you have done here, with all the explanation and how much of this was machine done and how much of this was human done.
2. `ckpt.pt` - final checkpoint (must contain the recorded step count, as the starter's `train.py` already does)
3. Your modified code, including a working `evaluate.py` + `tokenizer.py`
4. `RUNLOG.md` - one entry per training run: hypothesis, what changed, dev bpb before/after, what you concluded. **This is graded.**
5. `NOTES.md` - max 10 sentences: your best configuration and why it works.

Rules on tools

AI coding assistants (Claude Code, Codex, Cursor, Copilot) are **allowed**. We have measured what they score unassisted; your grade lives in what you add beyond that, in your RUNLOG reasoning, and in a discussion of your choices afterwards.



Suggested hour

- **0–10 min:** run the baseline (~3 min), score it on dev, read train.py and list everything questionable about it.
- **10–45 min:** experiment. Each run is ~3 min, so budget for 6 - 10 runs. Change ONE thing at a time and log it. Watch the loss curve, not vibes.
- **45–55 min:** best configuration, final full run, score, save.
- **55–60 min:** RUNLOG.md and NOTES.md.

A candidate who tries something ambitious (a new optimizer, an architecture swap), watches it lose, and can explain *why* it lost and fixes the loss, will score better with us than one who safely tuned nothing.

Good luck.